CODEBASE HEALTH AUDIT — NON-TECHNICAL FOUNDERS

Your Product's Safety and Scalability Check

This report tells you whether your codebase is **safe to scale, hand off, or take to investors**. Written in plain language with business impact first.

WHAT WAS SCANNED

Full-spectrum audit of the Hindsight codebase: security, architecture, code quality, scaling, maintainability, dependency health, and test coverage. Four-tier adversarial review (P0 → P1 → P2 → Pass spot-check) with deterministic scoring.

00 Executive Summary

No immediately exploitable vulnerabilities, but your API is wide open to abuse and one 7,000-line file is slowing your team down. Fix security defaults and start decomposing the god class before scaling.

Security scored 75/100 — no P0 blockers, but three findings compound: no CORS policy, no rate limiting, and raw error messages leak schema details to unauthenticated clients. Architecture is 81/100 — dragged down by a single 7,000-line file that handles everything. The good news: dependency health (90), testing (90), and code quality (94) are strong. The foundation is sound; it's the default configuration and structural debt that need attention.

WHAT THIS MEANS FOR YOU

Add CORS and rate limiting before exposing the API to the internet. Right now, any origin can call your API with unlimited frequency. Combined with the default no-auth mode and raw error messages, an internet-facing deployment would expose database schema details to anyone who asks.

Do not add a second engineer before addressing the 7,000-line MemoryEngine. Every change in that file risks breaking something unrelated. Unit

tests don't exist yet (the test seam does), and the file combines 8 distinct responsibilities. Clean architecture is what makes teams fast.

If you're preparing for investor diligence, fix SEC-03 (CORS + rate limiting), start the MemoryEngine decomposition (ARCH-01), and make torch optional (DEP-02). These three items will show up in any technical review. The rest can be phased.

Top Findings

WARNING MEDIUM CHANGE

SEC-03: Your API has no CORS policy and no rate limiting — any website can call it, as fast as it wants, with no throttle.

WARNING LARGE FEATURE OR MIGRATION

ARCH-01: MemoryEngine is a 7,000-line god class — every change risks breaking something unrelated. Needs decomposition.

WARNING MEDIUM CHANGE

SCA-01: No rate limiting on LLM calls — a burst of requests can exhaust your provider quota and budget.

WARNING MEDIUM CHANGE

TST-01: Core engine has zero dedicated unit tests. A 7,000-line file with no unit tests means refactoring is dangerous.

WARNING MEDIUM CHANGE

DEP-02: Every production install ships a 2GB AI library (torch) that most deployments never use. Remove it from defaults.

Remediation Plan at a Glance

3	11	9	3
Bugfix	Small Change	Medium Change	Large Feature or Migration

01 Health Score

B

OVERALL GRADE

84.9

HEALTH SCORE / 100

5

WARNINGS (P1)

21

PLAN ITEMS (P2)

Category Scores

Security	75/100	Maintainability	97/100
Architecture	81/100	Technical Debt	91/100
Code Quality	94/100	Dependency Health	90/100
Scaling	84/100	Testing	90/100

Weighted average: Security 25%, Architecture 20%, Code Quality 15%, Scaling 15%, Maintainability 10%, Technical Debt 5%, Dependency Health 5%, Testing 5%. Grade: ≥90 A, ≥75 B, ≥60 C, ≥50 D, <50 F.

02 Warnings (P1)

Five findings that need attention before scaling. Ordered by business risk.

SECURITY COMPLIANCE REPUTATIONAL → Technical details WARNING MEDIUM CHANGE

Your API has no CORS policy and no rate limiting — it's open to anyone, as fast as they want

What We Found: Your FastAPI application does not add CORS middleware or rate limiting. Any website can make requests to your API from a browser, and there's no limit

on how many requests a single client can send. This compounds with the default no-auth mode (SEC-01) and raw error messages (SEC-09) — together, any internet-facing deployment would expose database schema details to unauthenticated clients at unlimited speed.

Impact If Not Fixed: An attacker can scrape your API, exhaust your LLM provider quota (costing real money), and extract internal database details from error messages. If you handle any customer data, this is a compliance risk under GDPR and SOC2.

▶ ▶ AI Prompt — send this to Claude Code or Cursor

MAINTAINABILITY FINANCIAL

→ Technical details

WARNING LARGE FEATURE OR MIGRATION

Your core engine is a 7,000-line file that does everything — every change risks breaking something unrelated

What We Found: MemoryEngine is a single 7,000-line class with 97 methods that handles connection pooling, retain/recall/reflect orchestration, consolidation, bank CRUD, document management, entity resolution, and health checks. Changes to any one concern risk breaking every other concern. There are also zero dedicated unit tests for it (see TST-01).

Impact If Not Fixed: This is the #1 reason engineering slows down as your team grows. New developer onboarding extends from days to weeks. A bug fix in document management can silently break recall. The existing Interface ABC provides a clean decomposition starting point — but without unit tests as a safety net, refactoring is dangerous.

BEFORE CODING: PRODUCE A SOLUTION DESIGN DOCUMENT

This is a **Large Feature or Migration**. Before writing code, produce a solution design document referencing `architecture-review-hindsight-2026-05-27-technical.html`. The coding agent must load the full technical report for context before implementation.

▶ ▶ AI Prompt

SCALING FINANCIAL → Technical details

WARNING MEDIUM CHANGE

No rate limiting on LLM API calls — a burst of requests can exhaust your provider budget

What We Found: Your API has no rate limiting at either the HTTP or LLM call level. Internal semaphores cap concurrent requests (32 for LLM, 32 for recall, 5 for retain), but any single tenant can consume ALL slots. There's no per-tenant fairness and no per-second/per-minute rate limiting.

Impact If Not Fixed: A single user — or a script — can exhaust your LLM provider quota in minutes. Each GPT-4 call costs real money. Combined with no CORS and no authentication (SEC-03, SEC-01), anyone on the internet can drive up your cloud bill.

▶ ▶ AI Prompt

FINANCIAL SECURITY → Technical details WARNING MEDIUM CHANGE

Every production install ships a 2GB AI library that most deployments never use

What We Found: PyTorch (2GB+) and sentence-transformers are listed as required dependencies. Any production deployment — even those using external embedding providers like OpenAI — must install them. This inflates Docker image sizes by ~2GB and increases the security attack surface for every user.

Impact If Not Fixed: Slower deployments, higher storage costs, and an unnecessary attack surface for every customer who uses external embedding providers. Investors doing technical diligence will flag this as a sign that production readiness hasn't been prioritized.

▶ ▶ AI Prompt

MAINTAINABILITY FINANCIAL → Technical details WARNING MEDIUM CHANGE

Your 7,000-line core engine has zero dedicated unit tests — refactoring without a safety net

What We Found: The 81 test files all require a running PostgreSQL database. None mock the database — they're integration tests. MemoryEngine's 97 methods have zero dedicated unit tests. The MemoryEngineInterface ABC exists as a perfect mock seam, but it's completely unused for testing.

Impact If Not Fixed: This directly blocks the god class decomposition (ARCH-01). Without unit tests as a safety net, any refactoring of MemoryEngine is dangerous. You can't verify that behavior is preserved after extraction without spinning up a real database.

▶ ▶ AI Prompt

03 Plan Items (P2)

21 findings to address over time. Ordered by domain, then impact.

Security

SECURITY COMPLIANCE → Technical details PLAN SMALL CHANGE

Default mode allows unauthenticated API access

What We Found: DefaultTenantExtension returns a valid context without checking any credentials. This is documented as development-only, but there's no startup warning when it's active in production.

Impact If Not Fixed: A misconfigured production deployment silently serves all data without authentication. Add a startup WARNING log.

SECURITY → Technical details PLAN SMALL CHANGE

MCP authentication can be disabled with an environment variable

What We Found: A flag can disable MCP endpoint authentication. It's an explicit opt-in for MCP transports that handle their own auth, but there's no startup warning when enabled.

Impact If Not Fixed: Someone reviewing logs wouldn't see that auth bypass is active. Add a startup WARNING.

SECURITY → Technical details PLAN MEDIUM CHANGE

SQL table names built with string formatting, not proper escaping

What We Found: The function that builds qualified table names uses f-string interpolation across 5 modules (131 call sites). Currently safe because table names come from a hardcoded whitelist, but the pattern is a maintenance hazard — a future developer could pass user input.

Impact If Not Fixed: If someone adds a code path passing user input to `fq_table()`, it becomes SQL injection. Migrate to proper identifier quoting.

SECURITY → Technical details PLAN BUGFIX

Admin CLI uses string formatting for destructive SQL operations

What We Found: The admin CLI's TRUNCATE and REFRESH MATERIALIZED VIEW commands use f-string interpolation instead of proper identifier quoting. Only accessible locally, but TRUNCATE is destructive.

Impact If Not Fixed: Minimal attack surface (local admin only), but proper quoting is best practice for destructive operations.

SECURITY → Technical details PLAN SMALL CHANGE

Default database password in Helm chart

What We Found: The Helm chart defaults to password `hindsight` for PostgreSQL. Many deployments ship with defaults.

Impact If Not Fixed: An easily guessable database password in any deployment that doesn't override it. Make the password required or auto-generated.

SECURITY REPUTATIONAL → Technical details PLAN MEDIUM CHANGE

HTTP 500 error responses leak database schema details to clients

What We Found: 45 error handlers return `detail=str(e)`, which can include database table names, column names, and constraint names from `asyncpg.PostgresError`. Combined with no-auth and no-CORS defaults, this exposes internal details to anyone.

Impact If Not Fixed: Attackers can map your database schema from error messages. A single global exception handler can fix all 45 instances.

Architecture

MAINTAINABILITY → Technical details PLAN MEDIUM CHANGE

Business logic uses raw SQL strings — schema changes must be updated in two places

What We Found: MemoryEngine makes 99+ direct asyncpg SQL calls, while schema changes are managed by Alembic (SQLAlchemy). When a table changes, you need to update both the migration and the embedded SQL strings. No runtime dual pattern exists, but the coordination burden is real.

Impact If Not Fixed: Schema changes risk breaking runtime queries if the embedded SQL strings aren't updated. This would be resolved naturally by the Repository pattern in ARCH-01 decomposition.

MAINTAINABILITY → Technical details

PLAN LARGE FEATURE OR MIGRATION

The API module is a 4,200-line file mixing routes, models, and business logic

What We Found: `http.py` combines FastAPI route definitions, Pydantic models, auth middleware, and business logic in one file. It's well-organized internally, but hard to navigate and test in isolation.

Impact If Not Fixed: Developer velocity decreases as new endpoints must be added to an already sprawling file. Splitting into APIRouter-based modules would improve navigation and enable parallel development.

BEFORE CODING: PRODUCE A SOLUTION DESIGN DOCUMENT

This is a **Large Feature or Migration**. Before writing code, produce a solution design document referencing `architecture-review-hindsight-2026-05-27-technical.html`. The coding agent must load the full technical report for context before implementation.

MAINTAINABILITY → Technical details

PLAN MEDIUM CHANGE

Configuration file is 1,300 lines with 100+ manual environment variable lookups

What We Found: Each new config option requires edits in three places (ENV_ constant, DEFAULT_ constant, from_env() assignment). ConfigFieldAccessProxy is well-designed but sits atop an unwieldy monolith.

Impact If Not Fixed: Adding configuration is error-prone and slow. Group related settings into typed namespaces using pydantic-settings.

MAINTAINABILITY → Technical details

PLAN BUGFIX

Database helper function copied to 5 different files

What We Found: `fq_table()` is implemented independently in 5 modules with different signatures. 131 call sites depend on it. MemoryEngine's version uses context variables;

others take explicit parameters.

Impact If Not Fixed: If someone fixes a bug in one copy, the other four still have it. Extract to a single shared module.

Code Quality

MAINTAINABILITY → Technical details PLAN MEDIUM CHANGE

21 catch-all error handlers make it hard to distinguish real bugs from expected failures

What We Found: MemoryEngine contains 21 bare `except Exception` blocks. Each logs the error, but they make it difficult to tell whether a failure is expected or a new bug. Most are deliberate firewalls for extension error handling and operation cleanup.

Impact If Not Fixed: Debugging production issues becomes harder because real bugs look the same as expected failures in logs. Catch specific exception types instead.

MAINTAINABILITY → Technical details PLAN SMALL CHANGE

Inconsistent error handling across AI providers

What We Found: The OpenAI-compatible provider catches broad `Exception` in 4 places while the Anthropic provider catches specific exceptions everywhere. This inconsistency means some AI provider errors are harder to diagnose.

Impact If Not Fixed: Standardize by creating a common `LLMProviderError` base class with specific subtypes.

Scaling

SCALING → Technical details PLAN SMALL CHANGE

Database connection pool could bottleneck under heavy load

What We Found: The default pool max is 100 connections. Under high concurrency, the pool could be exhausted. No circuit breaker exists for database connection failures.

Impact If Not Fixed: Add connection pool metrics/logging and consider a circuit breaker pattern.

SCALING → Technical details PLAN SMALL CHANGE

Local AI models load synchronously at startup, blocking the server

What We Found: Embedding and reranking models load sequentially during initialization, adding 5-15 seconds to cold start. External embedding providers skip this entirely, but local deployments wait.

Impact If Not Fixed: Lazy initialization for the embedding model would improve cold start times for local deployments.

SCALING FINANCIAL → Technical details PLAN BUGFIX

No limit on how large a request body can be

What We Found: The API has no global request body size limit. Only file uploads have size constraints. A malicious client could send a 1GB JSON payload to exhaust server memory. Combined with no rate limiting (SEC-03), this amplifies the DoS surface.

Impact If Not Fixed: Add request body size limiting middleware. Configure limits via environment variables.

Technical Debt

TECHNICAL DEBT FINANCIAL → Technical details PLAN LARGE FEATURE OR MIGRATION

God class decomposition has been deferred indefinitely

What We Found: This is the technical debt tracking for ARCH-01. MemoryEngine combines 8+ responsibilities that should be separate modules. A phased decomposition plan exists: WorkerManager → DocumentManager → BankManager → ConnectionPool → ConsolidationEngine. MemoryEngine becomes a facade.

BEFORE CODING: PRODUCE A SOLUTION DESIGN DOCUMENT

This is a **Large Feature or Migration**. Before writing code, produce a solution design document referencing `architecture-review-hindsight-2026-05-27-technical.html`. The coding agent must load the full technical report for context before implementation.

TECHNICAL DEBT → Technical details PLAN MEDIUM CHANGE

Orchestration engine (25,000+ lines) has zero test coverage

What We Found: The cobuilder engine has 5,171 lines across core orchestration modules. Only the conditions subsystem has any tests. Pipeline parsing, session management, and signal protocol are untested.

Impact If Not Fixed: Any refactoring of the orchestration engine is risky without characterization tests. Prioritize tests for core paths before any changes.

MAINTAINABILITY → Technical details PLAN SMALL CHANGE

Auto-generated client code is checked into git, creating review noise

What We Found: Python and TypeScript client SDKs are auto-generated from OpenAPI specs but committed to git. CI checks for drift, but the checked-in code creates git diff noise and a maintenance burden.

Impact If Not Fixed: Either add to .gitignore and generate at build time, or add a strict CI drift check on every PR.

Maintainability

MAINTAINABILITY → Technical details PLAN SMALL CHANGE

No shared linting configuration enforced in CI across the monorepo

What We Found: The monorepo has 9 Python packages, each with its own linting config. A lint script exists but isn't run as a mandatory CI step. Code already passes ruff when run manually, but CI doesn't enforce it.

Impact If Not Fixed: Style drift across packages. Add ruff lint to CI with a shared root configuration.

Dependency Health

MAINTAINABILITY COMPLIANCE → Technical details PLAN SMALL CHANGE

CVE patch dependencies pinned directly instead of via constraints file

What We Found: Transitive dependencies like pyasn1, urllib3, and aiohttp are pinned in pyproject.toml with CVE fix comments. This works but creates maintenance burden when upper-level packages eventually include the fixes.

Impact If Not Fixed: Use a constraints file or add pip-audit/safety to CI for automated vulnerability detection.

Testing

MAINTAINABILITY → Technical details

PLAN SMALL CHANGE

No test coverage metrics or enforcement in CI

What We Found: CI runs pytest but with no `--cov` flag, no coverage threshold, and no coverage reporting. Critical modules have no unit test coverage baseline.

Impact If Not Fixed: Add pytest-cov and set minimum coverage thresholds for critical paths. This directly enables the god class decomposition by providing a safety net.

04 Remediation Plan

Prioritized by business risk. Phase 1 before your next internet-facing deploy. Phase 2 before your next milestone. Phase 3 as time permits.

Summary by Size

CATEGORY	COUNT	REQUIRES SOLUTION DESIGN
BUGFIX	3	No
SMALL CHANGE	11	No
MEDIUM CHANGE	9	Recommended
LARGE FEATURE OR MIGRATION	3	**Yes** — must produce solution design first

PHASE 1

Secure the API (Before Next Internet-Facing Deploy)

These compound into a real risk when the API is internet-reachable. Fix them this sprint.

- SEC-03: Add CORS middleware and rate limiting to FastAPI `MEDIUM CHANGE`
- SEC-09: Add global exception handler to sanitize 500 responses `MEDIUM CHANGE`
- SCA-04: Add request body size limiting middleware `BUGFIX`
- SEC-01: Add startup WARNING when DefaultTenantExtension is active `SMALL CHANGE`
- SEC-07: Make Helm PostgreSQL password required or auto-generated `SMALL CHANGE`

`PHASE 2` Structural Fixes (Before Next Milestone)

These affect how fast you can ship and how confident you can be in changes.

- TST-01: Add unit tests for MemoryEngine (unblocks ARCH-01 decomposition) `MEDIUM CHANGE`
- ARCH-01: Decompose MemoryEngine god class `LARGE FEATURE OR MIGRATION` — requires solution design first
- SCA-01: Add per-tenant rate limiting for LLM calls `MEDIUM CHANGE`
- DEP-02: Make torch/sentence-transformers optional via extras_require `MEDIUM CHANGE`
- ARCH-03: Split http.py into FastAPI APIRouter modules `LARGE FEATURE OR MIGRATION` — requires solution design first

`PHASE 3` Hardening & Polish (As Time Permits)

Important for long-term health but not blocking for the next milestone.

- SEC-02: Add startup WARNING when MCP auth disabled `SMALL CHANGE`
- SEC-04: Migrate fq_table() to proper identifier quoting `MEDIUM CHANGE`
- SEC-05: Add asyncpg quote_ident to admin CLI SQL `BUGFIX`
- ARCH-04: Refactor config into typed namespaced groups `MEDIUM CHANGE`
- ARCH-05: Extract shared fq_table() utility `BUGFIX`
- CQ-01: Replace bare excepts with specific exception types `MEDIUM CHANGE`
- CQ-02: Standardize LLM provider error handling `SMALL CHANGE`
- SCA-02: Add connection pool metrics and circuit breaker `SMALL CHANGE`
- SCA-03: Lazy-load embedding model at startup `SMALL CHANGE`
- MAI-01: Add shared ruff config and enforce in CI `SMALL CHANGE`
- TST-02: Add pytest-cov and coverage thresholds to CI `SMALL CHANGE`
- TD-02: Add characterization tests for cobuilder engine `MEDIUM CHANGE`
- TD-03: Move generated client code out of git or add strict CI check `SMALL CHANGE`
- DEP-01: Move CVE patches to constraints file `SMALL CHANGE`
- ARCH-02: Create Repository layer (natural outcome of ARCH-01 decomposition) `MEDIUM CHANGE`

05 What's Working Well

Seven areas passed inspection with no findings. These are strengths you can highlight.

PASS **Security — Credentials properly managed via environment variables**

All API keys, database URLs, and service account keys are loaded from environment variables with no hardcoded defaults. Sensitive fields are marked with `_CREDENTIAL_FIELDS` and never exposed via API.

PASS **Architecture — Clean bounded contexts for integrations and clients**

Integration packages (CrewAI, LiteLLM, PydanticAI, etc.) are properly separated with their own pyproject.toml, dedicated test suites, and minimal cross-dependencies. Client SDKs are auto-generated from OpenAPI specs.

PASS **Architecture — Good ABC interface for MemoryEngine public API**

MemoryEngineInterface (585 lines) properly defines the public contract with abstract methods, type hints, and comprehensive docstrings. This provides a clean decomposition starting point and enables future mock testing.

PASS **Code Quality — Good use of Pydantic models for API validation**

HTTP endpoints use Pydantic BaseModel, Field, and field_validator for request/response validation. The API contract is well-defined with type hints and validators.

PASS **Maintainability — Good documentation and docstring coverage**

Configuration has comprehensive docstrings. MemoryEngineInterface has thorough method-level documentation. SECURITY.md provides a vulnerability reporting policy.

PASS **Dependency Health — CI pipeline includes comprehensive multi-version testing**

CI tests Python 3.11 through 3.14, builds Docker images, runs integration tests, lint checks, and cross-client tests (Python, TypeScript, Rust, Go). Dependency security

patches are explicitly tracked.

PASS

Testing — Comprehensive CI testing across all client libraries

CI covers API (4 Python versions), all 4 client SDKs, integration tests, upgrade tests, and OpenAPI compatibility checks. This is thorough cross-platform validation.
